

Parallel architectures and APIs

COMP90025 Parallel and Multicore Computing
School of Computing and Information Systems
© The University of Melbourne

Udaya Parampalli

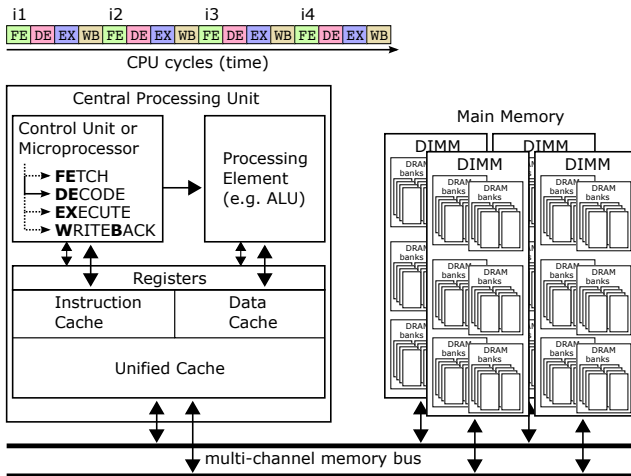
2026 Semester II

- 1 Parallelism on a single Processor
 - Processor and memory
 - Pipelining
 - Word size
 - Single Instruction Multiple Data
- 2 Between one and multiple processors
 - Co-Processor
- 3 Parallel Computer Architecture
 - Flynn's classifications of parallel architectures
 - Shared memory multiprocessor
 - Distributed memory
- 4 Programming Models
 - Approaches
 - Performance with a single core
 - Multithreading Model
 - Multiple Process Model

At the end of this module, you should be able to...

- explain the roles of the components of a parallel computer
- explain different types of parallelism
- choose between shared memory and distributed memory parallelism
- choose the right programming model for given hardware

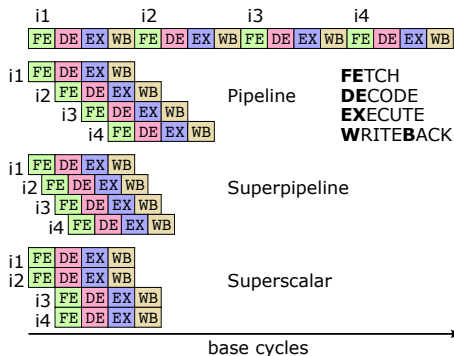
Elementary Components



- If the CPU is “hyper-threaded” it means that a separate set of registers are maintained for each of two threads and superscalar is used. Many people avoid hyper-threading today because of security concerns.

<https://www.wired.com/story/intel-mds-attack-speculative-execution-buffer/>

Parallelism from Pipeline, Superpipeline and Superscalar architectures



pipelining: Overlap different execution stages (different hardware)

superpipelining: Divide into smaller stages, with a faster clock speed

superscalar: Multiple pipelines in parallel

Word size

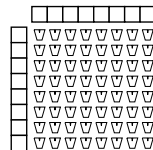
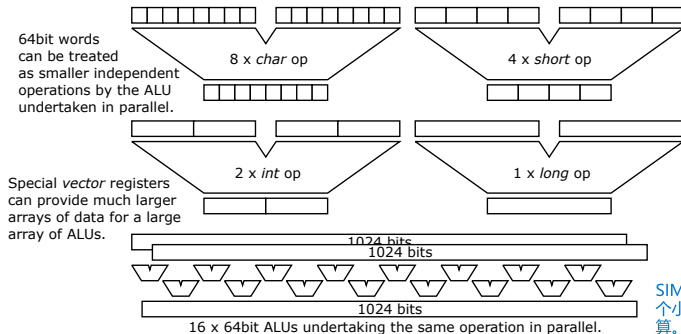
Year	Processor	Word Size, w	Integer	Float	Instruction	Address Res.
1974	Intel 8080	8	$w, 2w$	—	$w, 2w, 3w$	w
1975	ILLIAC IV	64	w	$w, \frac{1}{2}w$	w	w
1975	Motorola 6800	8	w	—	$w, 2w, 3w$	w
1976	Cray-1	64	$24, w$	w	$\frac{1}{4}w, \frac{1}{2}w$	w
1978	Intel 8086	16	$\frac{1}{2}w, w$	—	$\frac{1}{2}w, w, \dots, 7w$	8
1978	VAX	32	$\frac{1}{4}w, \frac{1}{2}w, 1\cdots 32$	$w, 2w$	$\frac{1}{4}w, \dots, 14\frac{1}{4}w$	8
1979	Motorola 68000	32	$\frac{1}{4}w, \frac{1}{2}w, w$	—	$\frac{1}{2}w, w, 7\frac{1}{2}w$	8
1985	Intel 80386	32	$\frac{1}{4}w, \frac{1}{2}w, w$	—	$\frac{1}{4}w, \frac{3}{4}w, 8\cdots 120$	8
1985	ARMv1	32	$\frac{1}{4}w, \frac{1}{2}w$	—	w	8
1985	MIPS	32	$\frac{1}{4}w, \frac{1}{2}w, w$	$w, 2w$	w	8
1992	Alpha	64	$\frac{1}{4}w, \frac{1}{2}w, w, 8$	$\frac{1}{2}w, w$	$\frac{1}{2}w$	8
1992	PowerPC	32	$\frac{1}{4}w, \frac{1}{2}w, w$	$w, 2w$	w	8
2003	x86-64	64	$\frac{1}{4}w, \frac{1}{2}w, w, 8$	$\frac{1}{2}w, w, 80$	$8\cdots 120$	8
2013	ARMv8/9-A	64	$\frac{1}{4}w, \frac{1}{2}w, w, 8$	$\frac{1}{2}w, w$	$\frac{1}{2}w$	8
TBD	RISC-V RV128I	32	$\frac{1}{4}w, \frac{1}{2}w, w, 2w, 4w$	$w, 2w, 4w$	8	

[https://en.wikipedia.org/wiki/Word_\(computer_architecture\)](https://en.wikipedia.org/wiki/Word_(computer_architecture)) – units in bits

The word size is a unit of measure of how much work gets done at once for various key aspects of the processor architecture.

- Integer – the size of an integer that can be processed in a single instruction
- Float – floating point number that can be processed in a single instruction
- ALU (Arithmetic/Logic Unit) – integer data processed in a single operation
- Instruction – the size of a single instruction
- Bus width – data transferred over an internal interconnection in one clock cycle

Single Instruction Multiple Data



String and text processing specialization. 64 simple ALUs provide character operations on all pairs of characters over two strings of length 8 characters.

SIMD是把一个宽的寄存器切成若干个小块，让alu对这些小块同时运算。

Pipelining时间上重叠不同指令的不同阶段

Superscalar同一时刻多条不同指令并发执行

SIMD同一条指令，同时作用在多份数据上

SSE 4.2 STTN1: $16 \times 16 = 256$ byte operations

1.2~13.8 speedup

If data is not stored as a contiguous array in memory then overheads for loading scattered data will be noticeable.

SSE and AVX data types

	<code>__m128</code>	float	float	float	float				
	<code>__m128d</code>	double				double			
	<code>__m128i</code>	byte	byte	byte	byte	byte	byte	byte	byte
	<code>__m128i</code>	short	short	short	short	short	short	short	short
	<code>__m128i</code>	int		int		int		int	
	<code>__m128i</code>	long long				long long			
	<code>__m128i</code>	double quad word							
<code>__mm256</code>		float	float	float	float	float	float	float	float
<code>__mm256d</code>		double		double		double		double	

In the x86 architecture, SSE provides 128 bit registers, AVX (released 2011) provides 256 bit registers, where the lower half of each register is the corresponding SSE register, and AVX-512 (released 2016) provides 512 bit registers (not shown in the figure).

2020—: AMX Matrix extensions

并不是主流芯片都是512

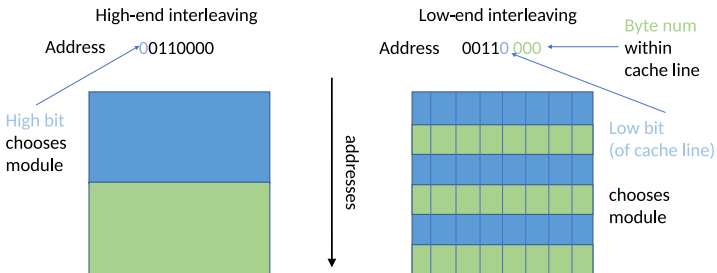
Addressing memory banks

Memory has pages (virtual memory) and cache (fast copies of recently used memory). Memory also comes in *modules*.

- There is a relatively long delay between requesting data and receiving it, compared to registers and immediate data.
- After a read, a memory module must “recharge” before it can be read again.

These limitations affect which addresses each memory bank should

contain. The aim is to ensure that memory modules are used in a round-robin order. A memory access pattern that repetitively access the same memory module may give noticeable performance losses.



Discussion questions I

Description: For two matrices A and B , both of size $n \times n$, compute $C = AB$. Let $n = 2^t$ for some $t \geq 0$ for convenience.

Time Complexity: $\Theta(n^3)$

Processors: 1

```

1: procedure MM1( $A, B, n$ )
2:   for  $i \leftarrow 0$  to  $n-1$  do
3:     for  $j \leftarrow 0$  to  $n-1$  do
4:        $C[i, j] \leftarrow \sum_{k=0}^{n-1} A[i, k]B[k, j]$ 
5:   return  $C$ 

```

Question (1): The recursive function is cache friendly. Explain why this is by examining the order of memory accesses. Compare this order to the cache unfriendly algorithm. Implement the algorithms in C and find out which algorithm runs the fastest for various increasing values of n .

Question (2): How much benefit do compiler optimization flags provide for these algorithms?

Description: For two matrices A and B , both of size $n \times n$, compute $C = AB$. Let $n = 2^t$ for some $t \geq 0$ for convenience.

Time Complexity: $\Theta(n^3)$

Processors: 1

```

1: procedure MM2( $A, B, n$ )
2:   if  $n = 1$  then
3:      $C[0] \leftarrow A[0] B[0]$ 
4:     return  $C$ 
5:   Let  $L = 0 \dots \frac{n}{2} - 1$  and  $U = \frac{n}{2} \dots n - 1$ 
6:   Let  $A_{ij}B_{kl} = \text{MM2}(A[i, j], B[k, l], \frac{n}{2})$ 
7:    $C[L, L] \leftarrow A_{LL}B_{LL} + A_{LU}B_{UL}$ 
8:    $C[L, U] \leftarrow A_{LL}B_{LU} + A_{LU}B_{UU}$ 
9:    $C[U, L] \leftarrow A_{UL}B_{LL} + A_{UU}B_{UL}$ 
10:   $C[U, U] \leftarrow A_{UL}B_{LU} + A_{UU}B_{UU}$ 
11:  return  $C$ 

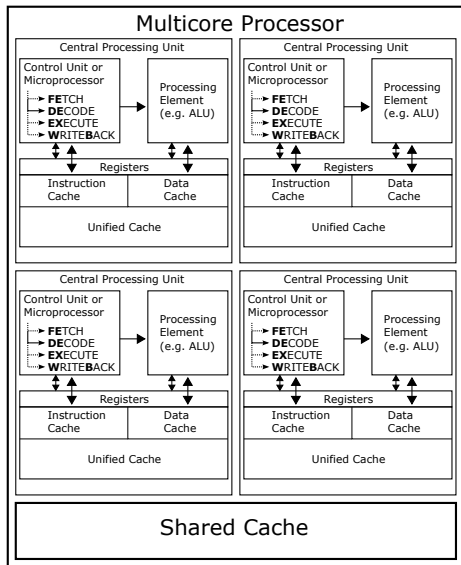
```

Discussion questions II

Compare the performance of `gcc` and `clang`, which are both available on Spartan.

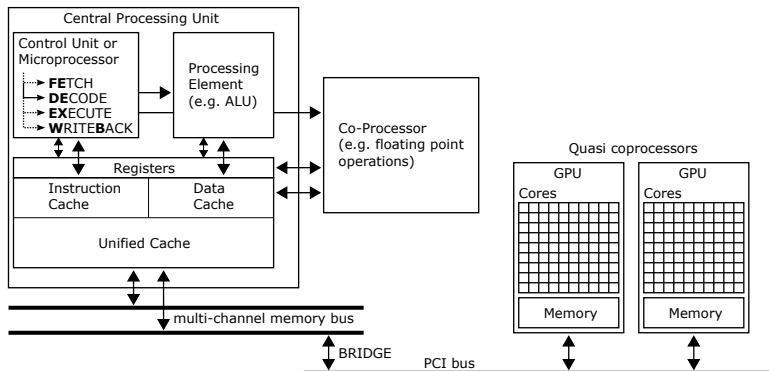
Question (3): Answer the questions above for matrices with different data types: `int`, `long long`, `float`, `double`.

Multicore Processor



- The CPU is replicated : **core**, within a multicore processor.
- The OS sees each core as a conventional CPU.
- Cores typically share an on-processor cache (Level 3).
- Yield limited
- Cores often have the same “instruction set architecture” (ISA), but different implementations.
 - **Performance** and **efficiency cores**.
- Can also have heterogeneous ISA cores: CPU, GPU, NPU
- **Servers (e.g., Xeon) are more likely to have only P-cores or only E-cores.**

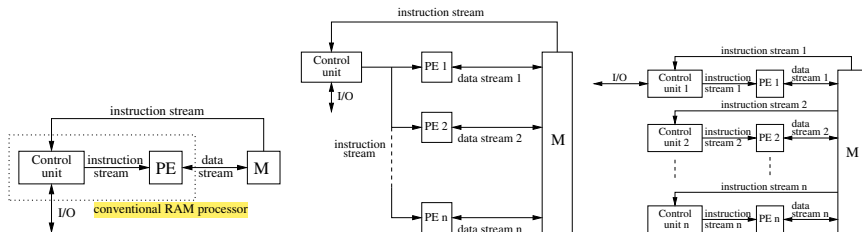
Co-Processor



A co-processor augments the processor with additional functionality.

- The traditional co-processor connects directly to the processor
- quasi co-processors are commonly provided via the PCI Bus.

Flynn's classifications of parallel architectures



- **SISD** – single instruction single data
- **SIMD** – single instruction multiple data
- **MISD** – multiple instruction single data
- **MIMD** – multiple instruction multiple data

ILLIAC IV — SIMD

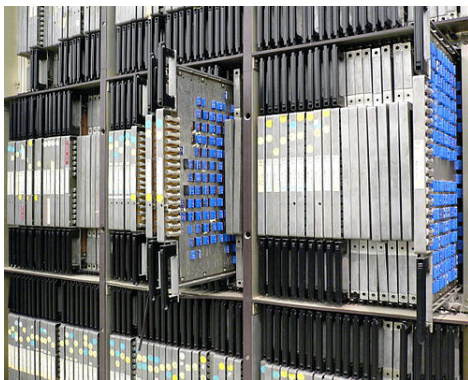


Figure: Illiac IV. Photo by Steve Jurvetson from Menlo Park, USA,
<https://www.flickr.com/photos/44124348109@N01>

The first SIMD machine built in 1972: designed to have 256 64-bit floating point units and 4 CPUs, it was eventually built with only 64 FPUs and a single CPU due to budget constraints.

Connection Machine CM-2 — MIMD

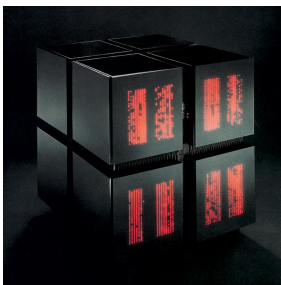


Figure: Connection Machine CM-2. Image © Thinking Machines Corporation, 1986. Photo: Steve Grohe.
From <http://www.mission-base.com/tamiko/cm/index.html>

The CM-2, released in 1987 by Thinking Machines Corporation, had 2^{16} PEs and up to 512MB of RAM. Even though each PE is only four bits wide, the machine could outperform many big Cray supercomputers on some specially programmed problems.

SIMD versus MIMD

SIMD

Vector operations: load/store ✓
data in bulk

smaller processing units ⇒ ✓
faster clock

Limited problems

Often harder to program

MIMD

Needs synchronization. Reads
more instructions

✓ More L2 cache if more cores
More expensive per PU

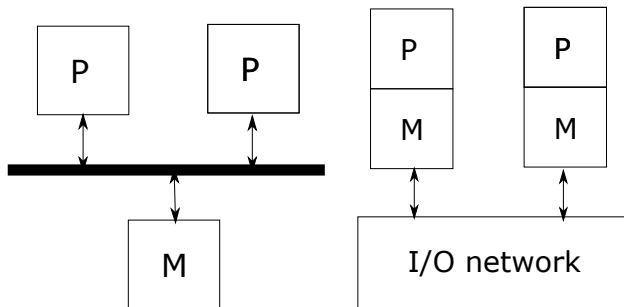
✓ More flexible

✓ Often easier to program

Modern processors are multicore, with each core providing some SIMD operations.

Shared and distributed memory

Big distinction: how memory connects to processors

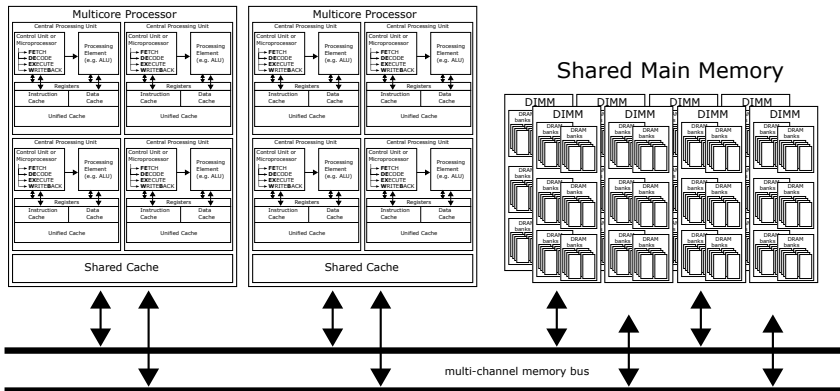


Memory	separate from processor
	load/store anywhere
Communicate	shared mem
	fast
Parallelism	thread

local to processor
load/store locally
explicit messages
slow
process

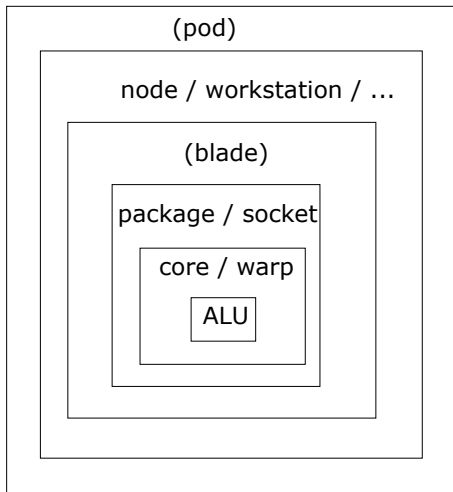
Symmetric shared memory multiprocessor

Multiple Processor Sockets



If the processors are identical, a shared memory computer is called a *symmetric multi-processor* or **SMP** machine.

A hierarchy of processors

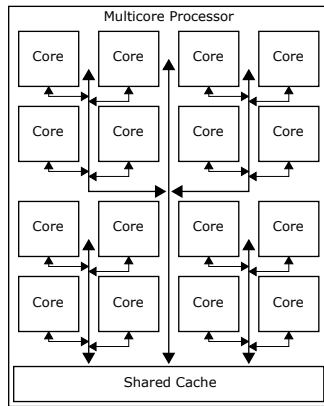
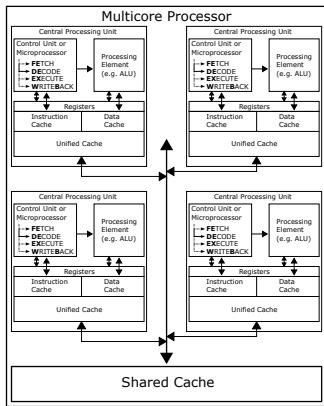


SMP limitations

Scalability limitations of bus-based SMP architectures

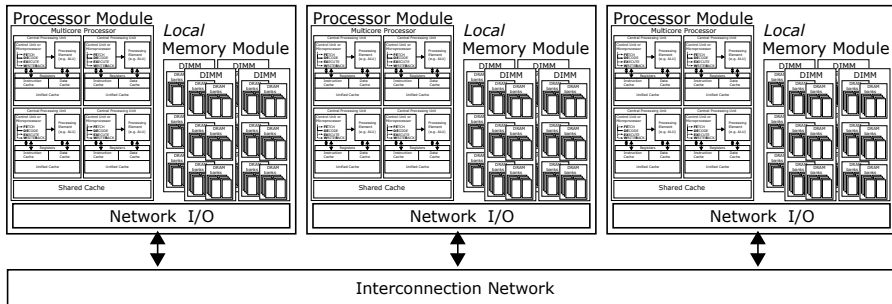
- Bus contention.
 - Larger on-processor caches
- Increasing bus channels has costs
- Longer buses are slower.
- Memory access symmetry.

Multicore design complexity



As the number of cores on a chip increases, the complexity of interconnection increases.

Distributed shared memory



Distributed shared memory (DSM) is no longer *uniform memory access* (UMA or symmetric) but rather *non-uniform memory access* (NUMA or asymmetric) because each processor module has a local memory module that it can access much faster than *remote* memory modules.

- Address space for each processor is distributed over its local and the remote memory modules, i.e. a single address space spans the modules.
- The DSM architecture can use buses, or interconnection networks.

Distributed memory (shared versus not shared)

DSM

Hardware transfers memory requests

Programmer unaware of where data is stored

Shared OS

Expensive specialized hardware

Distributed

Explicit message passing

Programmer keeps track of where data is stored

nodes can have different OSes

Cheaper, lots of commodity hardware

“multi-computer”

Frontier



Frontier, the HPE Cray EX235a built for Oak Ridge National Laboratory in the US, was the fastest supercomputer in the world, in July 2024. It has 8,699,904 cores, in AMD EPYC 64C 2GHz CPUs and AMD Instinct 250X GPUs, with a Slingshot-11 interconnect. It's Linpack benchmark score is 1,206.00 PFlop/s. It consumes 22,786.00 kW, at 62.86 GFlop/s/watt.

Spartan



Spartan has 6 racks of CPU nodes, 6 racks of GPU nodes and 2 racks of storage.

See <https://dashboard.hpc.unimelb.edu.au> for details. Spend some time reading the documentation on the web site.

Discussion questions

Question (4): From the first URL given below, a Spartan processor has 18 physical cores, so each node has 72 physical cores in total. The processor uses the Cascade Lakes architecture and a 20 core version of the processor architecture (Xeon Gold 6248, with more cores than the Gold6254 but slower clock speed) is shown in the second URL, where two such processors are shown connected together over an UltraPath Interconnect. Also the cache hierarchy is described, with respect to cores. Classify the architecture of the Spartan nodes and the partition overall. What is the purpose of the UltraPath Interconnect?

<https://ark.intel.com/content/www/us/en/ark/products/192451/intel-xeon-gold-6254-processor-24-75m-cache-3-10-ghz.html>

https://www.nas.nasa.gov/hecc/support/kb/cascade-lake-processors_579.html

Programming Models

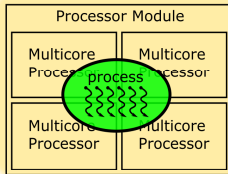
How do we write programs for the wide range of parallel architectures just described?

No one parallel programming model will be suitable for all of the architectures: at a high-level we might ask whether our parallel programming model is based on threads, i.e. a single process sharing memory, or multiple processes communicating by signals. If multiple processes, are the communication costs between nodes equal or unequal, and are they high or low compared to computation costs?

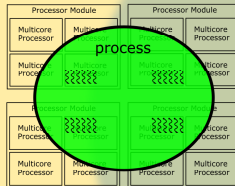
Processes and Threads

Shared Memory

Single Process
Multi-threading

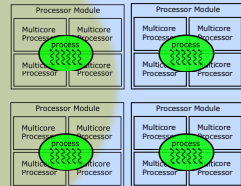


1 thread per core



1 thread per core

Multiple Processes
Single or Multi-threaded



Interprocess Communication required
1 single-threaded process per core
1 multi-threaded process per processor
1 multi-threaded process per module

Distributed

Implicit versus Explicit Programming Models

What decision are left to the programmer?

- *parallelism*: is a description of the parallel algorithm explicitly required? Do I need to write a parallel algorithm or can I simply provide a sequential algorithm and let the machine find the parallelism?
- *decomposition*: are the pieces of the parallel program explicitly defined? Do I need to define threads/processes or can it be left to the machine to determine that?
- *mapping*: are the pieces of the parallel program required to be explicitly mapped to different processors? Do I need to specify which threads/processes are run on which processors or does the machine distribute this for me?
- *communication*: is the communication between pieces of the parallel program required to be explicitly defined? Do I need to specify the synchronization between threads or the messages that are sent between processes?

Providing a completely implicit programming model, or automatic parallelism, is very unlikely due in part by **P**-complete problems.

Programmers are needed to problem solve and design explicit parallel solutions.

SPMD and MPMD

The terms *Single Program Multiple Data* (SPMD) and *Multiple Program Multiple Data* (MPMD) categorize the way in which pieces of a parallel program are implemented, obviously to the machine architecture, i.e., do not confuse this with Flynn's Classifications.

- In the SPMD model, each *task* is an identical instance of the program, and performs the same operations (maybe involving conditions) on a portion of the problem, usually based on a machine assigned identifier or *rank*. In a multi-threaded process, each thread can be thought of as a task.
- In the MPMD model, each task is an instance of potentially different programs, with task operations potentially based also on rank. E.g. one task may be a “master” task that coordinates work given to “worker” tasks, or tasks may form a pipeline or stream of processing stages or operators, such as in distributed stream processing.
`grep "double" *.c | more`

The most common model for high performance computing is SPMD. In cloud computing, it is usual to have different processes having different executables.

Considerations independent of parallelism

The choice of data structures can have a significant impact on **performance**.

- *Sequential memory accesses, and accesses to memory recently used*
- allocate all required memory at the beginning of the program, rather than dynamically allocating and releasing it.

Cache-friendly algorithms can provide an immense amount of performance improvement compared to algorithms that do not consider cache. Use of processor-specific registers and instructions are not generally portable, but may provide decent performance improvements.

- Consider data placement to facilitate vector loads and stores.

Compiler

The compiler should be considered. It may be better to allow the compiler to optimize unoptimized code, rather than manually optimizing code.

- Vendor supplied compilers for a given architecture should give better performance than than freely available compilers, and similarly less popular compilers may give worse performance. Differences in runtimes of up to 50% could be seen depending on whether compilers can detect and make use of optimized processor instructions. Always compare performance in any case.
- Compiler optimization flags should be used with care. Optimization may hide bugs, so debug both with and without optimization. Generally gcc's `-O2` flag can be used safely.

Automatic Vectorization

Compilers such as GCC can attempt to automatically detect vectorization opportunities in your sequential program:

- Your system hardware, compiler and OS must support it.
- Using SSE and AVX at the same time has some caveats. In this discussion we consider only AVX.
- Some commonly used flags, options and headers to investigate:
 - `-O3 -funroll-loops -fomit-frame-pointer -finline` for gcc optimization
 - `-march=native -mtune=native -mvzroupper` auto detect architecture, CPU and some AVX optimization
 - `-mavx` or `-mavx2` to use AVX
 - `#include <x86intrin.h>` for AVX/SSE extensions
 - `#include <bits/stdc++.h>` for main STD libraries
- Criteria for automatic loop vectorization:
 - The number of iterations in the loop must be constant once the loop begins.
 - The use of `break`, `continue`, external functions, and conditional flow in the loop may prevent automatic vectorization.
 - Dependencies between loop iterations can't be vectorized.

See “A Guide to Vectorization with Intel C++ Compilers” and for the GCC compiler see

<https://gcc.gnu.org/projects/tree-ssa/vectorization.html>

System and compiler capabilities

On Linux, a Kernel of 2.6.30+ is required and the hardware and compiler must support the vectorization operations.

```
$ gcc -march=native -dM -E - < /dev/null | egrep "SSE|AVX" |  
sort
```

```
#define __AVX2__ 1  
#define __AVX__ 1  
#define __MMX_WITH_SSE__ 1  
#define __SSE2_MATH__ 1  
#define __SSE2__ 1  
#define __SSE3__ 1  
#define __SSE4_1__ 1  
#define __SSE4_2__ 1  
#define __SSE_MATH__ 1  
#define __SSE__ 1  
#define __SSSE3__ 1
```

```
$ cat /proc/cpuinfo | egrep '(flags|model_name|vendor)' |  
    sort | uniq -c
```

```
2 flags : fpu vme de pse tsc msr pae mce cx8 apic sep  
mtrr pge mca cmov pat pse36 clflush mmx fxsr sse  
sse2 ht syscall nx rdtscp lm constant_tsc rep_good  
nopl xtopology nonstop_tsc cpuid tsc_known_freq pni  
pclmulqdq ssse3 cx16 pcid sse4_1 sse4_2 x2apic movbe  
popcnt aes xsave avx rdrand hypervisor lahf_lm  
abm 3dnowprefetch invpcid_single fsgsbase avx2  
invpcid rdseed clflushopt md_clear flush_l1d  
arch_capabilities
```

```
2 model name : 11th Gen Intel(R) Core(TM) i7-1165G7 @  
2 vendor_id : GenuineIntel
```

AVX Example

AVX programming uses AVX intrinsic functions, e.g. to convert:

```
multiply_and_add(const float* a, const float* b,  
                 const float* c, float* d) {
```

```
    for(int i=0; i<8; i++) {  
        d[i] = a[i] * b[i];  
        d[i] = d[i] + c[i];  
    }  
}
```

```
_mm256 multiply_and_add(_mm256 a, _mm256 b,  
                       _mm256 c) {  
    return _mm256_fmadd_ps(a, b, c);  
}
```

AVX Example

Another simple example:

```
int main() {  
    /* Initialize the two argument vectors */  
    __m256 evens = _mm256_set_ps(2.0, 4.0, 6.0, 8.0);  
                                10.0, 12.0, 14.0, 16.0);  
    __m256 odds = _mm256_set_ps(1.0, 3.0, 5.0, 7.0,  
                                9.0, 11.0, 13.0, 15.0);  
  
    /* Compute the difference between the vectors */  
    __m256 result = _mm256_sub_ps(evens, odds);  
  
    /* Display the elements of the result vector */  
    float* f = (float*)&result;  
    printf("%f_ %f_ %f_ %f_ %f_ %f_ %f_ %f\n", f[0], f[1],  
        f[2], f[3], f[4], f[5], f[6], f[7]);  
  
    return 0;  
}
```

AVX caveats

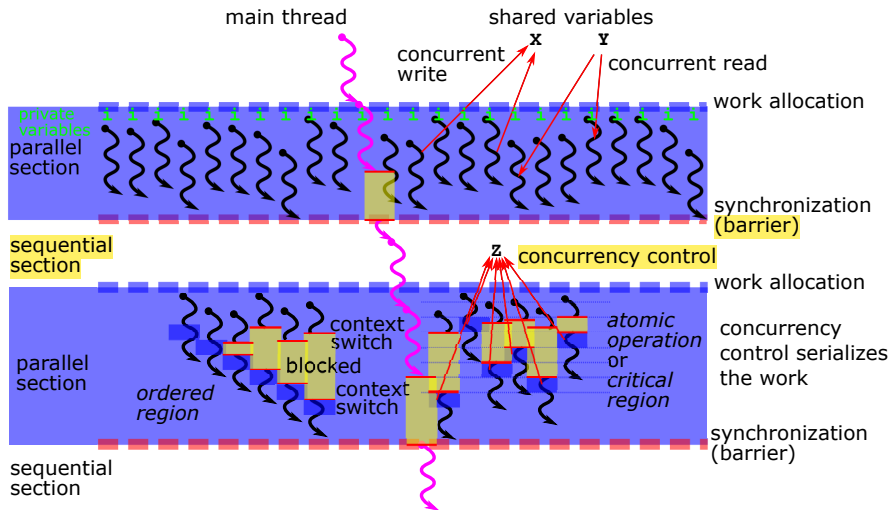
- AVX data loaded from memory must be memory aligned to the vector size or else an unaligned load must be used (aligned loads are faster):

```
float* aligned_floats = (float*)aligned_alloc(32,
                                                8 * sizeof(float));
// after data initialization
__m256 vec = _mm256_load_ps(aligned_floats);
```

- Intel's architecture is little-endian, e.g. the following prints 8 7 6 5 4 3 2 1 (use the setr version for "reverse"):

```
__m256i int_vector = _mm256_set_epi32(1, 2, 3,
                                       4, 5, 6, 7, 8);
int *p = (int*)&int_vector;
printf("%d %d %d %d %d %d %d %d\n",
       p[0], p[1], p[2], p[3], p[4], p[5], p[6], p[7]);
```

Single process, multi-threaded model



Multithreading considerations

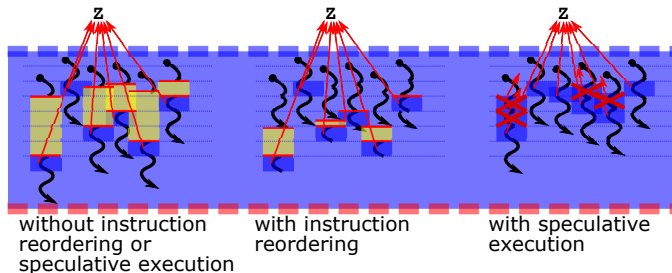
- For simple calculations and/or small sized problems, a single thread may well run faster than several threads due to the overhead of synchronization and cache interference (threads repetitively causing evictions of each other's data).
- Hyper-threading is turned off mostly these days, and this is the case on Spartan as well – one thread per core.
 - Using more than 1 thread per core may help to keep cores busy more of the time, depending on how thread synchronization is handled. However context switching is an overhead, in both time and memory consumption to store thread states. Performance will decrease with too many threads per core.
- Creation and destruction of threads is an overhead. Do not destroy threads until the end; leave them in an “idle pool”.

Multithreading considerations

Thread synchronization is expensive, especially between multicore processors (i.e. across sockets) compared to between cores on the same multicore processor, and compared to between threads on the same core. Try to limit the thread synchronization “complexity”.

- Avoid thread synchronization inside loops.
- Reduce synchronization by using more memory if necessary, e.g. an alternating “new” and “old” array of items, such that “new” is only being written to and “old” is only being read from.
- If you can specify thread affinity to cores and you know the expected synchronization patterns, i.e. which threads synchronize with others, try to minimize inter-core and inter-processor synchronization by being explicit about the thread to core mapping.

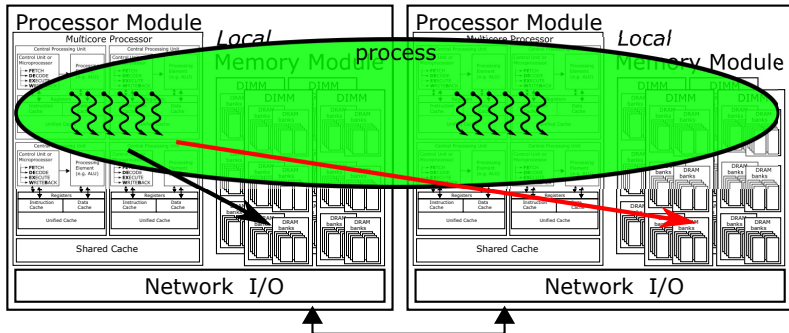
Instruction reordering and speculative execution



Instruction reordering allows a thread to continue with other instructions while it would have otherwise been blocked while waiting for access to a lock.

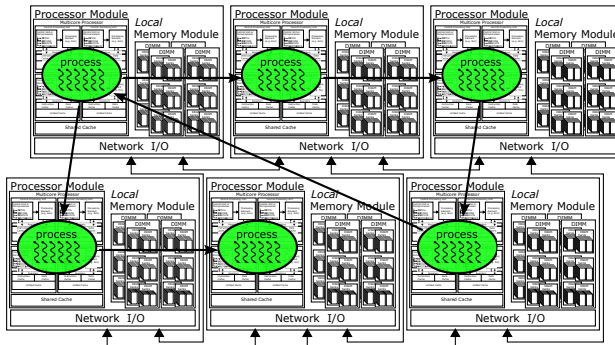
Speculative execution provides transactional memory constructs which executes operations optimistically in a contained (transaction) context and will rerun the transaction if a conflict was detected.

NUMA architecture considerations



When threads are allocated over a NUMA architecture the programmer must take care to understand the impact of thread-memory access and thread-thread synchronization. Transparent access to remote memory and remote threads is considerably more expensive than local memory.

Multiple Processes and Message Passing



Multiple processes are required in distributed architectures that don't permit threads of a single process to distribute over the architecture. In this case *message passing* is required between the processes. Note that if you can use multiple threads rather than multiple processes then you should – performance will be superior so long as you make appropriate use of OpenMP NUMA optimizations.

Message passing considerations

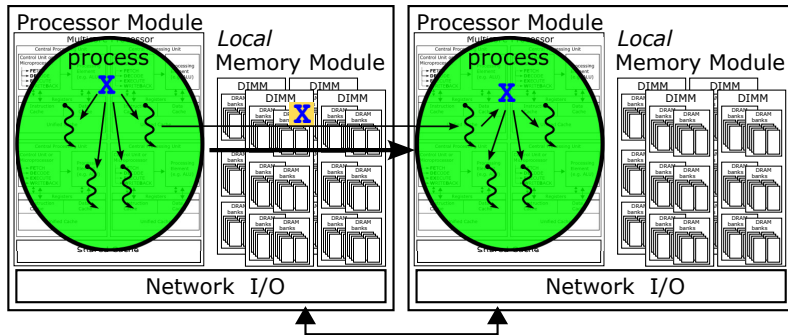
- It is harder to predict the performance of message passing models.
- Dynamically spawning new processes is expensive and should be avoided – create all processes once at the beginning of the program.
- Sending and receiving data can block threads.
 - Depending on the multi-threading model, some threads may or may not be allowed to send and receive data; e.g. perhaps only the main thread can do so.
- Data needs to be packed into messages and transmitted over a network.
 - Overheads for packing/unpacking complex data, both in terms of buffering capacity and cpu cycles.
 - Network communication latency is orders of magnitude greater than memory latency.

Message passing considerations

Communication patterns between n processes can become complex.

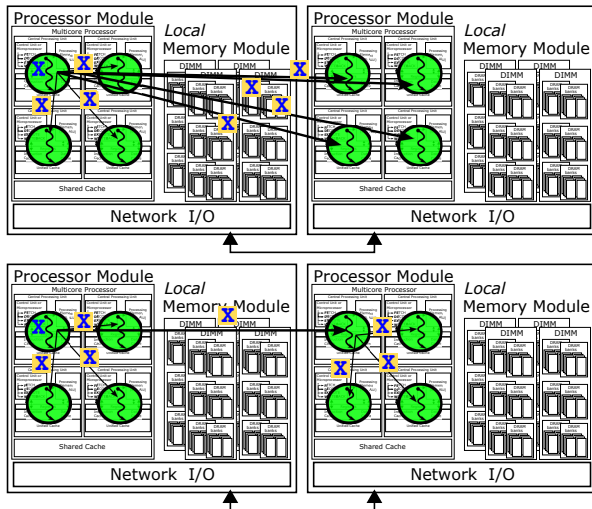
- $\mathcal{O}(n^2)$ communication patterns arise, e.g. for n -body calculations, and they can dominate the total running time of a parallel program if they are required often. Need to consider optimal message passing approaches.
- Available communication patterns may be limited by the network – affects our understanding of algorithm complexity.
- Message passing between massively parallel nodes (e.g. GPU nodes) can be quite inefficient – the GPUs can be waiting for data most of the time.

Message passing and multi-threading



A concurrent read of a shared memory element X , with one thread sending X to the other process, and then a concurrent write/read to distribute X to the threads of that process.

Single threaded multi-process



Discussion question

Question (5): Clearly the use of multiple processes with and without multi-threading leads to a large number of possible approaches for parallel program implementation. Is it ever reasonable to use many single-threaded processes rather than a single multi-thread process in cases where the machine architecture permits it? What are the pros and cons?

MPI – Message Passing Interface

Explicit: Message passing, parallelism, decomposition

Implicit: Mapping

For distributed memory systems

```
if(my_id == 0) { // master
    for(an_id = 1; an_id < num_procs; an_id++) {
        start_row = an_id*num_rows_per_process;
        MPI_Send( &num_rows_to_send, 1, MPI_INT,
            an_id, send_data_tag, MPI_COMM_WORLD);
        MPI_Send( &array1[start_row], num_rows_per_process,
            MPI_FLOAT, an_id, send_data_tag, MPI_COMM_WORLD);
    }
    for(an_id = 1; an_id < num_procs; an_id++) {
        MPI_Recv( &array2, num_rows_returned, MPI_FLOAT,
            MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
    }
} else {
    // slave process
}
```

Java 1.8+ parallelism

We do not use Java in this subject. However it is worthwhile noting that parallel constructs are available for multi-threading, including OpenMP support. While OpenMP is the mainstay in utilizing a multicore machine, Java 1.8 and above provides parallelism in much the same way, via streams and lambda functions:

```
int[] a = new int[100000];

IntStream.range(0, 100000)
    .parallel()
    .forEach(i -> { a[i]=2*i; }
);
```

Similarly to OpenMP, this is parallelism-explicit and mostly implicit in all other ways.

OpenCL

Open Computing Language is a framework for writing programs across heterogeneous platforms, e.g., consisting of multicore processors and coprocessors (GPUs, digital signal processors, etc).

// Multiplies $A \cdot x$, leaving the result in y .

// $A(i, j)$ is at $A[i \cdot \text{ncols} + j]$.

```
__kernel void matvec(__global const float *A,  
                    __global const float *x,  
                    uint ncols, __global float *y)  
{  
    size_t i = get_global_id(0);  
    __global float const *a = &A[i*ncols];  
    float sum = 0.f;  
    for (size_t j = 0; j < ncols; j++) {  
        sum += a[j] * x[j];  
    }  
    y[i] = sum;  
}
```